Algorithmes et programmation distribuée

Alexandrina KORNEVA

alexandrina.korneva@universite-paris-saclay.fr

Planning + Évaluation

Contenu des TP

> Threads
> Sockets
> Consensus

Évaluation

> Contrôle continu + TP noté

Threads

Concurrence vs. Parallélisme

Concurrence

Analogie: "Un chef, deux tâches."

Le chef commence la soupe. Pendant qu'elle mijote (attente E/S), il prépare la salade.

Résultat: Gérer plusieurs tâches à la fois.

Parallélisme

Analogie: "Deux chefs, deux tâches."

Un chef fait la soupe. Un autre chef fait la salade en même temps.

Résultat: Exécuter plusieurs tâches en même temps.

Processus vs. Thread

Processus

Un programme en cours d'exécution (ex: Chrome, VS Code).

Propriété clé: Lourd, et possède sa propre **mémoire isolée**.

Thread

Un "ouvrier" léger à l'intérieur d'un processus.

Propriété clé: Léger, et partage la **mémoire de son processus**.

Le partage de mémoire est ce qui rend les threads rapides, mais c'est aussi la source des problèmes

Le Problème: situation de compétition (*race condition*)

Deux threads (ou plus) essaient de lire et modifier la **même donnée** en **même temps**.

Analogie: Deux chefs lisent "stock = 1 carotte".

- Chef A prend 1 carotte.
- Chef B (qui a lu '1') prend 1 carotte.

Résultat: Le stock est à -1, mais les chefs pensent qu'il est à 0. Le résultat final est incorrect et imprévisible.

Situation de compétition (*race condition*)

Deux threads (ou plus) essaient de lire et modifier la **même donnée** en **même temps**.

Analogie: Deux chefs lisent "stock = 1 carotte".

- Chef A prend 1 carotte.
- Chef B (qui a lu '1') prend 1 carotte.

Résultat: Le stock est à -1, mais les chefs pensent qu'il est à 0. Le résultat final est incorrect et imprévisible.

DEMO

Les patterns

Démarrer

On crée un objet Thread et on appelle `start`:

```
import threading


def run (n,c):
    for i in range (n) :
        print (c)

t1 = threading.Thread (target = run , args = ( 100, '#') )
t2 = threading.Thread (target = run , args = ( 100, '!') )


t1.start ()
t2.start ()
```

Les patterns

Arrêt Propre

On ne "tue" pas un thread. On lui demande **poliment** de s'arrêter avec un signal (sentinelle)

```python
import threading
import time

def worker_bad():
    while True:
        print("Working...")
        time.sleep(1)

t = threading.Thread(target=worker_bad)
t.start()
```

Les patterns

Arrêt Propre

On ne "tue" pas un thread. On lui demande **poliment** de s'arrêter avec un signal (sentinelle)

```python
import threading
import time

def worker_good(stop_flag):
    while not stop_flag[0]:  # Vérifier le signal
        print("Working...")
        time.sleep(1)
    print("Stopping...")

stop_flag = [False]  # Liste partagée
t = threading.Thread(target=worker_good, args=(stop_flag,))
t.start()

time.sleep(3)
stop_flag[0] = True  # Signal d'arrêt
t.join()
```

Les patterns

Attendre (join)

Attend que le thread se termine. Sans .join(), le programme principal peut se terminer avant que le thread ait fini son travail.

```
import threading
import time

def worker_good(stop_flag):
    while not stop_flag[0]:  # Vérifier le signal
        print("Working...")
        time.sleep(1)
    print("Stopping...")

stop_flag = [False]  # Liste partagée
t = threading.Thread(target=worker_good, args=(stop_flag,))
t.start()

time.sleep(3)
stop_flag[0] = True  # Signal d'arrêt
t.join()
```

Le Pattern Essentiel: `with`

Gestion Sûre des Ressources

Utilisez `with` pour gérer tout ce qui doit être "fermé" (fichiers, sockets, locks...).

```
with open("mon_fichier.txt", "r") as f:
  # ... faire quelque chose avec f ...
```

Cela garantit que f.close() est appelé, **même si une erreur se produit**.

C'est une pratique essentielle.

Producteur-consommateur (producer-consumer)

Le problème: Comment organiser plusieurs threads qui travaillent ensemble?

La solution: Séparer en deux rôles

Producer (Producteur)
- Génère ou trouve du travail
- Met les tâches dans une file d'attente

Consumer (Consommateur)
- Prend les tâches depuis la file d'attente
- Les traite en parallèle

Exemples réels:
- Serveur web: recevoir les requêtes → les traiter
- Pipeline de données: lire les fichiers → les analyser
- Votre TP :)

File d'attente (`queue.Queue`)

N'utilisez pas une variable partagée. Utilisez une file d'attente.

Le module `queue.Queue` de Python gère tous les verrous (locks) pour vous. Il est **thread-safe**.

Producteur:

```
q.put(item)
```

Consommateur:

```
item = q.get()
```

TP

TP: Un Explorateur de Fichiers

Objectif du TP: Crawler & Workers

Le Producteur (Crawler)

Rôle: 1 seul thread.

Tâche: Utilise os.walk() pour scanner récursivement les dossiers.

Action: Met les filepath (chemins de fichiers) trouvés dans une queue.

Les Consommateurs (Workers)

Rôle: Plusieurs threads (ex: 4).

Tâche: Prennent un filepath depuis la queue.

Action: Ouvrent le fichier (avec with open(...)) et comptent les mots.